MIT | Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule University)

ASSIGNMENT 3

Design, Development and Deployment of a Web Application to Demonstrate Server-Side Validation Using ExpressJS.

Group Members

Name	PRN
Prince Bansal	202401100064
Pankaj Bharsakle	202401100067
Umang Ramteke	202401100068
Ayush Dudhe	202401100073

Aim

To design, develop, and test a responsive web application named **Labour Connect Express JS** using ExpressJS that facilitates worker profile registration, job applications, on-demand service booking, strict server-side form validation, RESTful API routing, structured HTTP status responses, and cloud deployment on Netlify.

Objectives

Technical Objectives

1. **RESTful Routing Architecture:** Implement clean, modular ExpressJS routing using *express.Router()* to handle client requests across dedicated endpoints (*/api/workers*, */api/jobs*, */api/applications*, and */api/service-requests*).
2. **Independent Server-Side Validation:** Enforce strict, manual server-side validation rules on incoming JSON payloads using native JavaScript conditions, regular expressions, and type guards without relying on client-side validation.
3. **HTTP Request and Response Handling:** Process client inputs using *express.json()* and *express.urlencoded()*, dispatching standard HTTP status codes (200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Internal Error) with structured JSON messages.

4. **Local Data Persistence:** Utilize lightweight in-memory JavaScript data arrays to ensure worker records, applications, and service requests persist across active server interactions.
5. **Serverless Cloud Deployment:** Configure and deploy the ExpressJS backend and React frontend to Netlify using *serverless-http* and Netlify Serverless Functions for cloud availability.

Functional Objectives

1. **Direct Marketplace Model:** Enable direct client-worker contact via phone (tel:) and service coordination with zero intermediary commissions.
2. **Interactive Directory Search & Filtering:** Allow users to filter workers by specialization, search dynamically across multiple fields (Name, Skill, Location), and apply for verified job openings.
3. **Responsive UI/UX & Live Test Bench:** Deliver a modern, accessible interface with Bootstrap alerts, preserving form inputs on validation failure, supplemented by an examiner test bench for viva demonstration.

Theory & Key Concepts

A. What is ExpressJS?

ExpressJS is an open-source, fast, unopinionated, minimalist web application framework for Node.js. It is designed for building robust web applications and RESTful APIs. Express provides a thin layer of fundamental web application features, such as routing, middleware support, and template rendering, without obscuring core Node.js capabilities.

B. Server-Side Validation vs. Client-Side Validation

Client-side validation provides immediate visual feedback in the browser, but it offers no real security because it can easily be bypassed using developer tools, cURL, or Postman. **Server-side validation** is mandatory as it forms an authoritative security barrier on the server. The Express backend treats all incoming payloads as untrusted, verifying every field for data type, presence, length, format, and logical consistency before processing or storing it.

C. HTTP Request and Response Lifecycle in ExpressJS

The lifecycle of a client request in Labour Connect Express JS follows a structured sequence:

- When a client submits an HTTP request, Express captures the incoming HTTP request stream.
- Middleware functions (such as *express.json()*) parse the request payload into JSON and populate *req.body*.

- Matching route handlers execute sequential validation algorithms. If any field fails, the server halts execution and returns HTTP 400 Bad Request with a clear error description.
- Upon successful validation, the data is pushed to in-memory arrays and the server returns HTTP 201 Created.

D. Routing & Middleware Architecture

ExpressJS organizes server functionality into modular routes and middleware functions:

1. **CORS Middleware:** Enables Cross-Origin Resource Sharing so the React frontend on port 5173 can securely communicate with the Express backend on port 5000.
2. **Express Router:** An instance of *express.Router()* isolates route definitions. To ensure compatibility across local servers and Netlify Functions, the router is mounted on */api*, */.netlify/functions/api*, and root paths.
3. **Error Handling Middleware:** Global 404 handlers capture unmatched endpoints, while centralized error handlers catch uncaught exceptions and return HTTP 500.

E. Server-Side Validation Rules and Implementation

Labour Connect implements precise validation rules directly inside POST route handlers:

- **Worker Registration (*POST /api/workers*):** Full name ≥ 2 characters; exactly 10-digit mobile number matching */^[0-9]{10}$/*; experience ≥ 0; non-empty location; expected payment > 0; availability strictly within predefined set (Available, Busy, Unavailable).
- **Service Request (*POST /api/service-requests*):** Customer name required; 10-digit mobile number; category in predefined trades list; valid non-past date (compared against server midnight date); description ≥ 10 characters.
- **Job Application (*POST /api/applications*):** Applicant name & skill required; 10-digit mobile number; job ID must exist in server in-memory jobs array.

F. Cloud Deployment via Netlify Serverless Functions

To deploy an Express application on Netlify's serverless edge infrastructure, *serverless-http* wraps the Express *app* instance into an AWS Lambda-compatible handler. The *netlify.toml* configuration file rewrites all incoming */api/** requests to */.netlify/functions/api/:splat*, ensuring identical API behavior in production without modifying backend source code.

Applications / Practical Use Cases

1. **Local Blue-Collar Employment Platform:** Connects informal, skilled workers (plumbers, carpenters, mechanics, electricians, painters, appliance technicians) directly with residential households and contractors.
2. **Zero-Commission Service Discovery:** Eliminates middleman aggregators by enabling direct telephone and service coordination between clients and tradespeople with server-side request confirmation.
3. **Civic & Municipal Directory:** Serves as a digital registry for local trade unions or community development boards to catalog verified local service providers.
4. **Academic Demonstration of Backend Engineering:** Showcases ExpressJS REST API routing, server-side data sanitization, HTTP status code protocol compliance, and decoupled full-stack architecture.

Software Requirements

Component	Specification / Tool
Operating System	Windows 10 / 11, macOS, or Linux
Runtime Environment	Node.js (v18.x or v20.x LTS recommended)
Package Manager	npm (Node Package Manager, v9.x or v10.x)
Backend Framework	ExpressJS (v4.19.2)
Middleware & Utilities	cors (v2.8.5), serverless-http (v4.0.0)
Frontend UI & Styling	React (v19.x), Bootstrap (v5.3.x), Bootstrap Icons
Build Tooling	Vite (v8.3.x)
Deployment Platform	Netlify (Static CDN + Netlify Serverless Functions)
Source Code Editor / IDE	Visual Studio Code (VS Code) with Node/ESLint extensions
Supported Web Browsers	Google Chrome (v110+), Microsoft Edge (v110+), Mozilla Firefox (v110+)

Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Processor (CPU)	Intel Core i3 / AMD Ryzen 3 (Dual-core, 2.0 GHz)	Intel Core i5 / AMD Ryzen 5 or higher
RAM (Memory)	4 GB RAM	8 GB RAM or higher
Hard Disk Space	2 GB free disk space (for node_modules and build output)	5 GB SSD free disk space

Display Resolution	1366 x 768 pixels	1920 x 1080 (Full HD)
Input Devices	Standard Keyboard and Mouse / Touchpad	Standard Keyboard and Mouse / Touchpad
Network Interface	Required for initial npm install packages	Broadband Internet connection

Code :-

https://github.com/umangramteke/labour-connect

https://1drv.ms/f/c/d6d4a9af10f46d53/IgCx2w3_LP_MRJ5GkIHVELwvAeEeLEGEbHj2uAcyELB6530?e=cnjZj5

Deployment Link

https://labour-connect1.netlify.app/

Conclusion

The **Labour Connect Express JS** web application successfully demonstrates the design, development, and deployment of a full-stack platform driven by rigorous server-side validation. Through modular ExpressJS routes, native JavaScript string and regular expression validation guards, explicit HTTP status codes (200, 201, 400, 404, 500), and custom JSON error dispatch, the backend completely safeguards application integrity independently of client-side logic. Comprehensive testing across diverse valid and invalid input scenarios confirmed that all constraints—including name length requirements, exact 10-digit phone verification, numeric bounds, non-past scheduling validation, and payload sanitization—function flawlessly. Successful deployment to Netlify utilizing *serverless-http* and Netlify Serverless Functions further proves cloud readiness and robust production viability.